Strategic Research Roadmap: Neurosymbolic Planning for Verifiable Autonomous System Remediation (NDSS 2027 Submission)

1. Introduction and Strategic Context

The intersection of artificial intelligence and cybersecurity has reached a critical inflection point, characterized by a paradox of capability versus reliability. While Large Language Models (LLMs) have demonstrated exceptional proficiency in threat detection, log analysis, and vulnerability explanation, their application to autonomous remediation—the act of modifying system state to fix vulnerabilities—remains fraught with risk. The central challenge, identified in the dissertation proposal as the "probabilistic correctness" problem, necessitates a fundamental shift in how autonomous agents are architected for high-stakes environments.[1] This document articulates a rigorous, evidence-based research plan designed to execute **Objective 1: To develop a Neurosymbolic Planning Framework for Verifiable Remediation**, targeting a submission to the Network and Distributed System Security (NDSS) Symposium 2027.

The urgency of this research is underscored by the widening "Speed of Action" gap between attackers and defenders. Adversaries leverage automation to exploit vulnerabilities within minutes of disclosure, while defenders, constrained by the manual verification of patches, often require days or weeks to remediate. The prevailing approach in current literature—relying on End-to-End (E2E) LLM generation for remediation scripts—is fundamentally flawed for critical infrastructure because it lacks verifiable safety guarantees. An LLM operating as a probabilistic token predictor cannot inherently guarantee that a generated rm -rf command will not inadvertently target a critical system directory due to a hallucinated variable expansion. To bridge this gap, this research proposes a novel **Logic-Generate-Verify (L-Gen-V)** architecture. This framework imposes a neurosymbolic bottleneck, forcing the opaque, probabilistic intent of the LLM into the transparent, deterministic structure of the Planning Domain Definition Language (PDDL) before any executable code is synthesized.

Targeting the NDSS Symposium aligns with the venue's historical emphasis on system-level security and practical implementation. Unlike conferences that prioritize pure machine learning metrics or theoretical cryptography, NDSS demands rigorous system design, robustness against realistic threat models, and reproducible artifacts. The anticipated submission deadline for the **NDSS 2027 Summer Cycle** is late April 2026, based on the stable historical patterns of previous cycles.[2] This creates a focused operational window of

approximately four months (January through April 2026) to formalize the domain, engineer the translation pipeline, validate the verification mechanisms, and conduct the empirical evaluation required for a Tier-1 security publication.

This report serves as a comprehensive operational roadmap. It deconstructs the high-level objectives into granular technical milestones, defines the theoretical underpinnings of the proposed PDDL domain, and establishes the evaluation protocols necessary to demonstrate superiority over stochastic baselines. By integrating insights from recent advancements in automated program repair (APR) and formal methods—specifically leveraging tools like **Fast Downward**, **VAL**, and **SMT solvers**—this plan charts a path toward a seminal contribution in the field of verifiable autonomous defense.

2. Theoretical Framework and Literature Analysis

To construct a robust research plan, one must first rigorously define the theoretical foundations upon which the proposed solution rests. The failure of current "Agentic AI" in cybersecurity is not merely an engineering issue but a theoretical mismatch between the problem domain (which is logical and state-based) and the solution domain (which is statistical and semantic).

2.1 The Limitations of Stochastic Generation in Remediation

Current state-of-the-art approaches, such as **AutoPentest** [4] or generic **ReAct** (Reasoning and Acting) agents, model remediation as a sequence of text generation tasks. The agent observes a vulnerability description, retrieves relevant context, and generates a command (e.g., a Bash script or Ansible playbook) to fix it. While this approach benefits from the LLM's vast pre-training on code repositories, it suffers from intrinsic stochasticity. An LLM lacks an internal "world model" that tracks state transitions. It does not "know" that deleting a configuration file in step 1 makes it unavailable for editing in step 2; it only predicts that the token edit is statistically likely to follow delete in its training corpus.

This limitation manifests as "hallucination," where the model invents non-existent package names, flags, or dependencies.[1] In a software development environment, a hallucinated variable leads to a compilation error—a low-cost failure. In a production system environment, a hallucinated command can lead to data loss or service denial—a catastrophic failure. This distinction necessitates the move from **Generative AI** to **Neurosymbolic AI**, where the neural component handles the ambiguity of natural language and the symbolic component enforces the correctness of state transitions.

2.2 The Role of PDDL in System Modeling

The Planning Domain Definition Language (PDDL) provides the necessary formalism to bound the agent's behavior. Originating from the International Planning Competition (IPC), PDDL separates the **Domain** (the invariant physics of the world) from the **Problem** (the specific

initial state and goal).[5]

For the proposed research, the "Domain" is the Linux Operating System. Unlike the simplistic "Blocksworld" domains used in early AI planning research [7], the System Administration domain is complex, involving diverse object types (files, users, services, ports, packages) and predicates that capture their relationships (e.g., (depends_on service package)).

- **Types and Hierarchy:** The domain must utilize PDDL 2.1 features like :typing to define a hierarchy where configuration_file is a subtype of file, and system_user is a subtype of user.[8]
- **Predicates as State:** System state is captured not by variable values but by true/false predicates. For example, the security of an SSH configuration is not represented by the file content string, but by the abstract predicate (secure_ssh_config host). This abstraction is crucial; it decouples the *intent* (make SSH secure) from the *implementation* (editing /etc/ssh/sshd_config).
- **Action Schemas:** The core innovation in Objective 1 is defining the (:action...) schemas for system remediation. An action like install_package is not just a command string; it is a logical transition that requires (not (installed?p)) as a precondition and provides (installed?p) as an effect.

This formalization allows us to cast vulnerability remediation as a **Classical Planning Problem**. Given a vulnerable initial state (e.g., (vulnerable_log4j host)) and a secure goal state (e.g., (not (vulnerable_log4j host))), the planner searches the state space for a valid sequence of actions. Crucially, the planner *cannot* find a plan that violates the domain physics. If the domain specifies that a service cannot be restarted unless its config file exists, the planner will never propose a plan that deletes the config before the restart. This provides the **correctness-by-construction** property that raw LLMs lack.

2.3 Advancements in Neurosymbolic Translation

Recent literature has demonstrated the viability of using LLMs to translate natural language into PDDL. The **LLM+P** framework [7] established that LLMs are significantly better at translating problems into PDDL for external solvers than they are at solving the problems directly. Similarly, **PDDL-Instruct** [9] showed that fine-tuning LLMs on "Logical Chain-of-Thought" data—where the model explains the preconditions and effects of actions before generating PDDL—improves translation accuracy.

However, a critical gap remains: existing frameworks focus on robotics or logic puzzles. There is no extant literature applying LLM-to-PDDL translation for **cybersecurity remediation**. The vocabulary of system administration (e.g., "iptables," "chmod," "systemctl") is vastly different from "pick up block." Objective 1 aims to close this gap by developing the first SysAdmin PDDL domain and the associated translation mechanisms. This involves not just translation, but **State Abstraction**—using tools like osquery to capture the low-level system state and map it

to high-level PDDL predicates.[11]

2.4 Formal Verification of Scripts

The final component of the theoretical framework is the verification of the "concretized" script. Once the planner produces a symbolic plan (e.g., step 1: update_config, step 2: restart_service), this must be translated into executable Bash or Ansible code. This step re-introduces some risk of translation error. To mitigate this, the research will leverage **Satisfiability Modulo Theories (SMT)** solvers like **Z3**.[12] By converting the generated script logic into SMT formulas, we can check for satisfiability against safety constraints. Furthermore, tools like **ShellCheck** [14] provide static analysis for syntax safety, while symbolic execution engines like **KLEE** [15] can explore execution paths to detect potential crashes or unsafe memory operations in binary utilities invoked by the script.

3. Detailed Research Architecture and Methodology

The proposed system architecture is a closed-loop control system, distinct from the open-loop generation typical of chatbots. This section details the technical specifications of the **Logic-Generate-Verify (L-Gen-V)** loop.

3.1 Component A: The SysAdmin Domain Engineering

The foundational task is the creation of sysadmin.pddl. This file defines the "physics" of the remediation environment. It must be expressive enough to capture relevant security states but abstract enough to prevent state-space explosion.

Design Specification:

- **Language Level:** PDDL 2.1 (supporting numeric fluents if necessary, though STRIPS is preferred for solver efficiency).[5]
- **Object Types:** host, file, directory, user, group, service, package, port, firewall_rule.
- **Predicates:**
  - **State:** (file_exists?f), (service_running?s), (port_open?p), (user_exists?u).
  - **Security:** (file_owned_by?f?u), (permission_is?f?perm), (vulnerable?pkg), (config_parameter_set?f?param?value).
  - **Relationships:** (depends_on?s?pkg) (service s requires package pkg), (configures?f?s) (file f is the config for service s).
- **Actions:**
  - edit_file_parameter: Changes a value in a config file. Precondition: file exists. Effect: parameter is set.
  - service_restart: Restarts a service. Precondition: service is installed. Effect: service state refreshed.
  - iptables_block: Adds a firewall rule. Precondition: iptables installed. Effect: traffic blocked.

- update_package: Upgrades a package. Precondition: network available. Effect: package version updated, (not (vulnerable?pkg)).

The challenge of **State Explosion**—where listing every file on a Linux system creates a problem file too large for any planner—must be addressed via **Dynamic Scoping**. The system will not ingest the entire filesystem state. Instead, a pre-processing step using the LLM will identify the *relevant scope* (e.g., "This CVE relates to nginx and /etc/nginx/nginx.conf") and only instantiate those specific objects in the PDDL problem file.[16]

3.2 Component B: The Neurosymbolic Translator

This component creates the bridge between the unstructured CVE report and the structured PDDL problem.

Workflow:

1. **Ingestion:** The system receives a CVE description (e.g., "CVE-2021-44228: Log4j vulnerability via JNDI lookup").
2. **Context Retrieval:** It queries the live system using osquery to check for the presence of the vulnerable component (e.g., SELECT * FROM packages WHERE name LIKE '%log4j%';).
3. **Translation:** An LLM (fine-tuned or prompted with ICL) receives the CVE text and the osquery JSON output. It generates the problem.pddl file:
   - **Objects:** Instantiates log4j-core as a package.
   - **Initial State:** (:init (vulnerable log4j-core) (package_installed log4j-core)...)
   - **Goal State:** (:goal (not (vulnerable log4j-core)))
4. **Validation:** The generated PDDL is passed to a syntax validator (e.g., VAL). If invalid, the error log is fed back to the LLM for **Iterative Refinement**.[17]

3.3 Component C: The Planner and Concretizer

The Planner: We will utilize **Fast Downward**, a highly optimized classical planner.[7] It takes the sysadmin.pddl and the generated problem.pddl as input. It outputs a symbolic plan: (update_package log4j-core), (restart_service java-app).

The Concretizer: A secondary LLM call translates these symbolic steps into executable code.

- **Input:** Symbolic Plan ((update_package log4j-core)).
- **Prompt:** "Translate this PDDL action into an Ansible task for Ubuntu 20.04."
- **Output:**
  YAML
  ```yaml
  - name: Update Log4j
    apt:
      name: log4j-core
      state: latest
  ```

This separation of concerns ensures that the LLM is not "inventing" the remediation logic, but merely "translating" a verified plan into syntax.

3.4 Component D: The Verification Shield

Before the Ansible playbook is executed, it passes through the "Shield."

1. **Syntax Checking:** ansible-lint and ShellCheck ensure the code is syntactically valid.[14]
2. **Safety Invariant Verification:** We formalize critical safety rules (e.g., "SSH port must remain open"). We check if the generated plan or script violates these. For high-assurance, we can map the script logic to SMT formulas (e.g., using **Z3**) to prove satisfiability of the safety constraints.[13] For example, if the script modifies sshd_config, the SMT solver checks if the resulting configuration logically permits root login if the policy forbids it.
3. **Sandbox Simulation:** The script is executed in a disposable Docker container (the "Digital Twin") to verify it runs without error and actually remediates the vulnerability (checked via a PoC exploit) before being approved for the live system.[19]

4. Evaluation Strategy: SysRepair-Bench

A significant contribution of this research will be **SysRepair-Bench**, a novel benchmark dataset tailored for autonomous system remediation. Existing benchmarks like *SWE-bench* focus on software engineering (modifying source code to fix bugs), which is distinct from system administration (modifying configuration and state).[20]

4.1 Benchmark Composition

We will construct SysRepair-Bench by adapting and filtering scenarios from **VulnRepairEval** [21] and **LinuxFLBench**.[20] The selection criteria will prioritize:

1. **Configuration Vulnerabilities:** Issues fixable by changing config files (e.g., sshd_config, nginx.conf, pg_hba.conf).
2. **Dependency Management:** Issues fixable by upgrading/removing packages.
3. **Permissions/Access:** Issues fixable by chmod, chown, or usermod.

The benchmark will consist of **50 reproducible scenarios**, each encapsulated in a Docker container. Each scenario will include:

- **The Vulnerable State:** A Dockerfile setting up the vulnerable environment.
- **The Threat Context:** The CVE description and associated metadata.
- **The Success Oracle:** A verify.sh script that attempts to exploit the vulnerability (the "PoC") and checks for system functionality (Regression Test). The remediation is successful *only* if the PoC fails AND the system functionality test passes.

4.2 Metrics

The evaluation will compare the proposed Neurosymbolic Framework against a baseline **ReAct Agent** (e.g., AutoGPT or LangChain with GPT-4).

- **Proof of Remediation (PoR):** Percentage of scenarios where the vulnerability is successfully neutralized.
- **Safety Violation Rate:** Percentage of runs where the agent breaks the system (e.g., deletes critical files, stops essential services) or violates a safety invariant.
- **Plan Optimality:** The length and cost of the remediation plan (shorter is generally better/safer).
- **Hallucination Rate:** The frequency with which the agent attempts to invoke non-existent commands or reference invalid file paths.

5. Execution Plan (January – April 2026)

This section outlines the strategic execution path, broken down into monthly phases with SMART goals (Specific, Measurable, Achievable, Relevant, Time-bound). The timeline targets the **NDSS 2027 Summer Cycle** submission deadline in late **April 2026**.

Phase 1: Foundation & Formalization (January 2026)

Goal: Establish the theoretical core (PDDL Domain) and the experimental testbed (SysRepair-Bench).

- **Week 1-2: PDDL Domain Engineering**
  - **Task:** Develop the sysadmin.pddl domain file.
  - **Details:** Define types (file, service, package, user), predicates (active, installed, configured), and actions (install, remove, edit, restart, chmod).
  - **Milestone (M1.1):** A valid sysadmin.pddl file passing syntax checks in the VAL validator, capable of modeling at least 20 distinct system operations.
- **Week 3-4: Benchmark Curation**
  - **Task:** Create SysRepair-Bench.
  - **Details:** Fork **VulnRepairEval** and **CVE-Bench**. Filter for "Infrastructure" and "Configuration" CWEs. Create 50 Dockerized scenarios. Write the verify_fix.sh oracle for each.
  - **Milestone (M1.2):** 50 reproducible containers where the vulnerability can be verified and the fix can be manually applied and tested.
  - **Milestone (M1.3):** Develop the state_to_pddl.py tool using osquery to dump the container state into a PDDL :init block with <1s latency.

Phase 2: Pipeline Development (February 2026)

Goal: Build the Neurosymbolic Agent (Translator + Planner + Concretizer).

- **Week 5-6: LLM Translator & Fine-Tuning**

    - **Task:** Implement the Text-to-PDDL translation.
    - **Details:** Create a dataset of (CVE Description + System State) -> (PDDL Problem) pairs. Use this to prompt-engineer GPT-4 (ICL) or fine-tune a Llama-3-8B model (using LoRA) to generate valid PDDL.
    - **Milestone (M2.1):** The LLM achieves >85% syntactic validity on generating PDDL problem files for the training set.
- **Week 7: Planner Integration & Feedback Loop**
    - **Task:** Integrate **Fast Downward** and the **Iterative Refinement** loop.
    - **Details:** Build the wrapper that runs the planner. If the planner fails (e.g., "Goal unreachable"), parse the error and feed it back to the LLM to adjust the PDDL problem definition.[17]
    - **Milestone (M2.2):** A fully automated script remediate.py that takes a CVE string and outputs a symbolic plan.
- **Week 8: Concretization & Verification Shield**
    - **Task:** Implement the Plan-to-Code translation and Safety Checks.
    - **Details:** Map symbolic actions to Ansible/Bash. Integrate **ShellCheck** for syntax validation. Implement the SMT-based safety check (e.g., asserting that the plan does not violate (active sshd)).
    - **Milestone (M2.3):** The pipeline outputs verified, executable scripts.

Phase 3: Evaluation & Analysis (March 2026)

Goal: Conduct experiments, gather data, and perform ablation studies.

- **Week 9-10: End-to-End Evaluation**
    - **Task:** Run the full SysRepair-Bench suite.
    - **Details:** Execute the Neurosymbolic Agent on all 50 scenarios. Record success/failure, time taken, and safety violations.
    - **Milestone (M3.1):** Complete data set for the proposed method.
- **Week 11: Baseline Comparison**
    - **Task:** Run the Baselines.
    - **Details:** Execute a standard ReAct Agent (GPT-4o + Tools) on the same 50 scenarios.
    - **Milestone (M3.2):** Comparative data showing the Neurosymbolic approach has a lower "Safety Violation Rate" (even if raw success rate is similar).
- **Week 12: Ablation Studies & Safety Stress Test**
    - **Task:** Analyze component contributions.
    - **Details:** Run tests *without* the Verification Shield. Run tests *without* the Planner (Direct Gen). Specifically test "Adversarial" prompts (Objective 3 preview) to see if the PDDL layer blocks malicious injections.
    - **Milestone (M3.3):** Quantitative evidence of the "Shield's" value.

Phase 4: Writing & Submission (April 2026)

Goal: Synthesize findings into the NDSS submission.

- **Week 13: Drafting Core Sections**
  - **Task:** Write Methodology, System Design, and Evaluation.
  - **Details:** Produce high-quality architecture diagrams (L-Gen-V Loop). Visualize the SysRepair-Bench results (Bar charts for Success/Safety).
- **Week 14: Drafting Intro & Related Work**
  - **Task:** Position the paper.
  - **Details:** Write a strong Introduction framing the "Probabilistic Correctness" problem. Write a nuanced Related Work section distinguishing the approach from *LLM+P* (robotics) and *AutoPentest* (offense).
- **Week 15: Review & Refinement**
  - **Task:** Internal Peer Review.
  - **Details:** Get feedback from advisor/colleagues. Check against NDSS formatting guidelines. Ensure rigorous anonymization.
- **Week 16: Final Polish & Submission**
  - **Task:** Submit to HotCRP.
  - **Deadline: April 22, 2026** (Estimated).

6. Risk Management

Risk	Probability	Impact	Mitigation Strategy
State Explosion: The PDDL problem becomes too large for the planner (e.g., modeling every file on disk).	High	Severe	**Scope Abstraction:** Use the LLM to pre-filter relevant files (e.g., "Only include config files in /etc/ssh/"). Only instantiate objects in the PDDL problem that are strictly necessary for the specific CVE.[16]
Translation Failure: The LLM generates invalid	Medium	High	**Grammar-Constrained Decoding:** Use llama.cpp

PDDL that the planner cannot parse.			grammars (GBNF) to force the LLM output to strictly adhere to PDDL syntax, guaranteeing parseability.[22]
Benchmark Difficulty: Reproducing 50 CVEs takes too long.	Medium	Medium	**Prioritization:** Focus on the "Top 20" most critical configuration CVEs first. Quality of analysis trumps quantity of scenarios.

7. Conclusion

This research plan outlines a strategic, high-velocity path to delivering a Tier-1 security paper for NDSS 2027. By formalizing the problem of system remediation as a planning task rather than a generation task, this work addresses the fundamental trust deficit in AI-driven operations. The **Logic-Generate-Verify** architecture transforms the probabilistic output of LLMs into verifiable, deterministic plans, offering a solution to the "Hallucination Problem" in critical infrastructure. The execution of this plan, particularly the development of SysRepair-Bench and the sysadmin.pddl domain, will establish a new standard for **Verifiable Autonomous Defense**.

Table 1: Key Tools & Resources

Component	Tool / Library	Role	Source
Planner	**Fast Downward**	Symbolic planning engine.	[7]
LLM	**Llama-3** / **GPT-4**	Semantic translation (Text -> PDDL).	[23]

Verification	**VAL / Z3 (SMT)**	Plan validation and safety invariant checking.	[9]
Static Analysis	**ShellCheck**	Syntax safety for generated scripts.	[14]
Benchmark	**VulnRepairEval**	Source for reproducible CVEs.	[21]
State Capture	**osquery**	Serializing system state to PDDL predicates.	[11]

Works cited

1. Dissertation Proposal Autonomous System Remediation.pdf
2. Security and Privacy Conference Deadlines, accessed December 29, 2025, https://sec-deadlines.github.io/
3. NDSS Symposium 2026 Call for Papers, accessed December 29, 2025, https://www.ndss-symposium.org/ndss2026/submissions/call-for-papers/
4. CVE-Bench: Benchmarking LLM-based Software Engineering Agent's Ability to Repair Real-World CVE Vulnerabilities - ACL Anthology, accessed December 29, 2025, https://aclanthology.org/2025.naacl-long.212.pdf
5. Planning Domain Definition Language - Wikipedia, accessed December 29, 2025, https://en.wikipedia.org/wiki/Planning_Domain_Definition_Language
6. Planning Domain Definition Language(PPDL) - GeeksforGeeks, accessed December 29, 2025, https://www.geeksforgeeks.org/artificial-intelligence/what-is-planning-domain-definition-language-ppdl/
7. Cranial-XIX/llm-pddl - GitHub, accessed December 29, 2025, https://github.com/Cranial-XIX/llm-pddl
8. PDDL Domain - Planning.wiki, accessed December 29, 2025, https://planning.wiki/ref/pddl/domain
9. PDDL-Instruct: Logical CoT Framework - Emergent Mind, accessed December 29, 2025, https://www.emergentmind.com/topics/pddl-instruct-framework
10. Enhancing Symbolic Planning Capabilities in LLMs through Logical Chain-of-Thought Instruction Tuning - Pulkit Verma, accessed December 29, 2025, https://pulkitverma.net/assets/pdf/vlfms_lm4plan25/vlfms_lm4plan25.pdf
11. clear-nus/gpt-pddl - GitHub, accessed December 29, 2025, https://github.com/clear-nus/gpt-pddl
12. Formal Verification with SMT Solvers: Why and How - UT Austin Computer Science, accessed December 29, 2025,

https://www.cs.utexas.edu/~moore/acl2/seminar/2009.04.29-johnson/final.pdf
13. SMT Models for Verification Debugging - ETH Zürich, accessed December 29, 2025, https://ethz.ch/content/dam/ethz/special-interest/infk/chair-program-method/pm/documents/Education/Theses/Cedric_Stoll_MA_report.pdf
14. ShellCheck – shell script analysis tool, accessed December 29, 2025, https://www.shellcheck.net/
15. Nix - KLEE, accessed December 29, 2025, http://klee-se.org/nix/
16. Leveraging Environment Interaction for Automated PDDL Translation and Planning with Large Language Models - arXiv, accessed December 29, 2025, https://arxiv.org/html/2407.12979v2
17. An LLM-Modulo Framework for Automated PDDL Domain Model Generation, accessed December 29, 2025, https://elib.uni-stuttgart.de/bitstreams/40946f84-8ab6-4934-8648-7d4b59103b7a/download
18. Ansible playbooks — Ansible Community Documentation, accessed December 29, 2025, https://docs.ansible.com/projects/ansible/latest/playbook_guide/playbooks_intro.html
19. From CVE Entries to Verifiable Exploits: An Automated Multi-Agent Framework for Reproducing CVEs - arXiv, accessed December 29, 2025, https://arxiv.org/html/2509.01835v1
20. Benchmarking and Enhancing LLM Agents in Localizing Linux Kernel Bugs - arXiv, accessed December 29, 2025, https://arxiv.org/html/2505.19489v1
21. VulnRepairEval: An Exploit-Based Evaluation Framework for Assessing Large Language Model Vulnerability Repair Capabilities - arXiv, accessed December 29, 2025, https://arxiv.org/html/2509.03331v1
22. ggml-org/llama.cpp: LLM inference in C/C++ - GitHub, accessed December 29, 2025, https://github.com/ggml-org/llama.cpp
23. The Ultimate Guide to Fine-Tune LLaMA 3, With LLM Evaluations - Confident AI, accessed December 29, 2025, https://www.confident-ai.com/blog/the-ultimate-guide-to-fine-tune-llama-2-with-llm-evaluations